

MiniRHUB — 40-Day Build Plan

A daily-target schedule to take the MiniRHUB project from empty repo to portfolio-grade in 40 days, following the phase order and "done-when" gates from the master design doc.

Read this before Day 1

The honest framing. The design doc recommends a *12-week* cadence (≈ 60 working days). This plan compresses that into 40 days. That is achievable, but only at a serious, near-full-time pace — assume **6–8 focused hours on most days**. If you have fewer hours, don't abandon the plan: use the off-ramps below, which still leave you "ahead of most new hires."

The non-negotiable rule carries over: you are simulating a job, not doing a tutorial. Every day's work ships the way a team ships — RFC or ADR where it matters, small PRs, a cold self-review the next morning, tests in CI, observability per PR, a runbook for each failure mode. The repo is the proof.

Two natural off-ramps (so the plan degrades gracefully):

- **End of Day 10** — a real trade flows in, normalized $\rightarrow$ enriched $\rightarrow$ observable. You're already ahead of most candidates.
- **End of Day 20** — the full end-to-end EMIR pipeline plus the distributed-systems-correctness story (idempotency, replay, DR drill). This is the single highest-signal milestone in the whole project. If you only reach here, you have the interview-winning sentence.

Structure. 40 days, grouped into 8 blocks of 5. Days 1–5 run the **domain crash course in parallel** (≈ 30 min/day mapping reading to a pipeline stage). Build EMIR first, the "New" event type first, Python first; C++ comes at the end where you can prove a real number.

Daily ritual: morning = pull 1–3 small scoped tasks; build with tests + a metric + a log line carrying `correlation_id`; open a PR; next morning, cold-review your own diff before merging. Friday of each block: a 2-minute demo capture + a three-line retro.

Block 1 — Days 1–5: Foundations + domain (Phase 0)

Goal of the block: a stack that stands up healthy in one command, a canonical model you can defend, and a lifecycle you can recite from memory.

- **Day 1 — Stack up.** Monorepo skeleton; `(docker-compose.yml)` bringing up PostgreSQL, Cassandra, Redis, Kafka, RabbitMQ, Comdb2; a health-check integration test

asserting all six are reachable; CI skeleton green on an empty service. (+30 min domain.) *Done when:* all six stores/brokers come up healthy in one command and CI is green.

- **Day 2 — Canonical model.** The canonical trade JSON schema; the lifecycle event types (New / Modify / Cancel / Correction); schema-validation unit tests. Write the ADRs: datastore choices, event-sourcing, envelope format, Kafka-vs-RabbitMQ split. (+30 min domain.)
 - **Day 3 — Operating model.** Repo layout (`/services`, `/libs`, `/docs`, `/tools`, `/regulatory`); branch protection on `main` (PRs required, CI must pass, one approving review — your reviewer-self); CODEOWNERS, PR template, issue templates, Projects board (Backlog → Ready → In Progress → In Review → Done), meaningful labels. Write your first RFC. (+30 min domain.)
 - **Day 4 — Observability skeleton + first incident.** `/health` + `/metrics` on a stub service; first Grafana dashboard; the structured-log + `correlation_id` pattern. Incident drill: bring a datastore up misconfigured, then write the "stack won't start" runbook. (+30 min domain.)
 - **Day 5 — Domain consolidation + buffer.** Be able to recite the pipeline and lifecycle from memory. Write the one-pager that separates **transaction reporting (RTS 22, confidential, T+1, via ARM)** from **trade transparency (RTS 1/2, public, via APA)** — the classic newcomer confusion. Catch up on anything from Days 1-4. *Block done when:* you can recite the lifecycle, CI is green on an empty service, and the stack is one command.
-

Block 2 — Days 6-10: Ingestion + Reference data (Phases 1-2)

Goal: real-world trade input lands on the bus in order with bad input quarantined, and trades emerge legally complete.

- **Day 6 — FIX venue simulator.** A FIX venue simulator streaming execution reports (QuickFIX in Python).
- **Day 7 — Ingestion gateway.** FIX → canonical JSON → Kafka `trades.raw`; idempotent publish (dedup key on exec-id); Kafka keyed by trade-id for per-trade ordering.
- **Day 8 — Harden ingest + incident.** Malformed-input policy → route to DLQ with reason codes (never crash). Property test: the mapper round-trips every supported FIX tag. Integration test: 10k synthetic trades land ordered per trade-id. Dedup test. Metrics + a Grafana panel. Incident drill: feed malformed FIX, diagnose from metrics, confirm quarantine, write the postmortem. *Done when:* 10k FIX trades land as ordered canonical messages, malformed input is quarantined with reason codes, and a postmortem is on file.

- **Day 9 — Reference data.** Reference tables in PostgreSQL (LEI, instruments/ISIN, counterparties, UPI); enrichment service that checks Redis first and falls back to PG on a miss.
 - **Day 10 — Refresh + cache + incident.** Daily refresh job modeling the authoritative sources (LEI=GLEIF, UPI=ANNA-DSB, instruments=ESMA FIRDS); cache-hit/miss unit tests; integration test that trades emerge with LEI/ISIN/UTI/UPI populated; cache-hit-rate SLI on a dashboard. Incident drill: stale reference data / cache stampede. ★ **Milestone:** a trade flows in, normalized and fully enriched, visible on dashboards.
-

Block 3 — Days 11-15: Eligibility DSL + Validation/Transformation + Submission (Phases 3-5)

Goal: which regimes apply is decided by *rules* not hardcoded `(if)s`; only correct, well-formed reports leave; reports go out and verdicts come back.

- **Day 11 — Rules DSL.** A small rule language (e.g. `WHEN asset_class = "RATES" AND counterparty.jurisdiction IN ("EU") THEN report_under("EMIR")`); RFC for the grammar; basic evaluation.
 - **Day 12 — Hot-reload + incident.** Hot-reload mechanism (rule edits take effect live, no redeploy); rule versioning; golden tests per rule; a test proving a live edit takes effect without restart. Incident drill: deploy a bad rule live → wrong regime → roll it back hot → postmortem.
 - **Day 13 — Validation.** Schema + business-rule validation rejecting with **reason codes**; the reason-code catalog; tests that bad trades NACK with the right codes.
 - **Day 14 — Transformation + incident.** Canonical → ESMA EMIR ISO 20022 XML validated against the **official XSD**; golden-file XML tests (byte-stable, XSD-valid); regression tests against ESMA validation rules. Incident drill: a schema-invalid edge case slips through → tighten validation → add the regression test that would have caught it. *Done when:* a canonical trade becomes schema-valid EMIR XML and bad trades NACK with proper reason codes.
 - **Day 15 — Submission core.** Mock Trade Repository accepting files over SFTP and returning ACK/NACK; submission service with a retry/backoff state machine over RabbitMQ (`(PENDING → SUBMITTED → ACKED | NACKED → RETRYING → DLQ)`); RFC for the state machine and backoff policy; idempotency so a retry never double-reports.
-

Block 4 — Days 16-20: Submission hardening + the correctness story (Phase 5 finish + non-negotiables)

Goal: the single highest-signal week — the distributed-systems-correctness story the role is really asking for.

- **Day 16 — Submission proofs.** Property test: N submissions of one trade yield exactly one accepted report. Chaos test: kill the TR mid-batch and prove every trade reaches a terminal state after recovery. Submission state dashboards (state counts, ACK/NACK rates, retry depth, DLQ size).
 - **Day 17 — Idempotency & delivery.** Layer the non-negotiables onto the slice: idempotency keys in Redis with idempotent writes everywhere; at-least-once delivery + dedup; per-trade ordering preserved (Kafka keyed by trade-id); dead-letter queues + replay.
 - **Day 18 — Backpressure & tracing.** Backpressure so the firehose can't topple a slow downstream. Write up the exactly-once-*effects*-vs-exactly-once-*delivery* distinction crisply. Distributed tracing threading `correlation_id` from ingest to submission.
 - **Day 19 — First two incidents.** Inject and resolve two real incidents with blameless postmortems — e.g. TR starts NACKing 100% (schema drift) → accuracy SLO alert fires → diagnose via golden-diff → fix behind a versioned transformer.
 - **Day 20 — DR/replay drill.** Wipe a downstream store, replay the event log, and prove the system rebuilds state *without double-reporting*. ★ **Milestone:** full end-to-end EMIR pipeline + the correctness story. If you stop here, you're already interview-ready.
-

Block 5 — Days 21-25: Lifecycle depth + audit & reconstruction (Phase 6)

Goal: every state change is permanent, and any trade's whole life is reconstructable on demand.

- **Day 21 — Lifecycle events.** Modify / Cancel / Correction; out-of-order handling (a Cancel arriving before its New → sequence or quarantine, never corrupt state).
- **Day 22 — Event store.** Write every state transition to Cassandra as an append-only log; ADR for the event schema; partition by trade-id + time; WORM-style immutability enforcement.
- **Day 23 — Config store + determinism.** Move the rules/config store onto Comdb2; replay-determinism test (folding the event log reproduces the exact current state).
- **Day 24 — Reconstruction.** A BVault-lite module linking each trade to its (simulated) communications/files and rebuilding the full timeline; a reconstruction test that produces a bundle.
- **Day 25 — Audit drill.** Incident drill: an "audit request" — reconstruct one trade's full life (trade + linked comms + every event) on demand and produce the bundle. Add

event-write-throughput and reconstruction-latency observability. *Done when:* you can replay one trade's entire life and produce a reconstruction bundle.

Block 6 — Days 26-30: Reconciliation + exceptions + multi-tenancy (Phases 7, 8, 10)

Goal: prove reports landed and your side matches the counterparty's; rejections triage themselves; tenants are isolated.

- **Day 26 — Pairing.** PySpark job over the Cassandra event store that pairs the two sides of EMIR trades and flags mismatches; RFC: what counts as a break, matching tolerances, pairing on UTI.
- **Day 27 — Recon + incident.** Three-way reconciliation (received vs reported vs acknowledged); customizable client extracts; a golden recon report against a fixture with deliberately seeded breaks (unmatched trade, field mismatch, timing break) — all must be caught. Incident drill: silently drop 0.5% of submissions upstream → prove the completeness recon catches the gap.
- **Day 28 — Exception management.** NACK triage (cluster/classify rejection reasons); anomaly detection on submission volumes (a sudden drop = a possible completeness gap); resubmission tooling. Tests + incident drill (inject a volume drop → completeness alert fires → resubmission flow used).
- **Day 29 — Multi-tenancy.** Per-tenant config, data segregation, entitlements; encryption in transit (TLS/SFTP) and at rest; immutable access/audit trail; a STRIDE-style threat-model RFC.
- **Day 30 — Isolation drill.** Cross-tenant access test that must fail; audit-trail completeness test; client onboarding flow; incident drill: attempt a cross-tenant read → prove isolation holds and the attempt is logged. ★ **Milestone:** depth complete — two tenants run side by side with zero data bleed.

(Phase 9 — BTCA-lite best-execution analytics — is optional at this pace. If you want the surveillance angle, slot a TCA module scoring executions against arrival-price/VWAP benchmarks here or after Day 40.)

Block 7 — Days 31-35: Framework/SDK + regulatory-change epic + start C++ (Phases 11, 12, 1')

Goal: the plumbing is a real library with an SDK, and the system survives a real regulatory migration.

- **Day 31 — Shared core.** Extract envelope/serialization/idempotency/retry/metrics/tracing into a shared `rhuh-core` library; semver + compatibility-policy ADR; contract tests guarding consumers against breaking changes.
 - **Day 32 — SDK + bindings.** A client SDK a firm uses to submit trades and poll status; a sample client app that submits a trade via the SDK; expose a C++ component to Python via pybind11. *Done when:* two services depend on `rhuh-core` and a sample client submits a trade through the SDK.
 - **Day 33 — Regulatory-change epic.** Version schemas, rules, and report formats pinned to regime go-live dates; run an old and a new (REFIT-style) format **in parallel** behind a feature flag; cut over by effective date with an audit trail of which format produced which report. RFC: the migration plan and dual-run mechanism.
 - **Day 34 — Migration proof.** Test that the same trade produces the correct format for its effective date, and that the migration runs without downtime. (This is the most domain-authentic phase — give it room.)
 - **Day 35 — C++ rewrite begins.** Start the hot-path rewrite (Phase 1'): rewrite the FIX gateway + routing/enrichment core in C++ (QuickFIX++), exposed to Python via pybind11; build the **benchmark harness** (warm-up, fixed dataset, percentiles not averages).
-

Block 8 — Days 36–40: C++ finish + UI + scale + portfolio polish (Phases 1', 13, 14)

Goal: a real latency win with numbers, live SLO dashboards, a non-engineer-usable UI, and a repo that needs no explanation.

- **Day 36 — Benchmark the win.** Finish the C++ rewrite; produce a repeatable before/after benchmark report (the target sentence: "p99 dropped from 8ms to 0.9ms at 50k msg/s"), ideally with a flame graph.
- **Day 37 — Client/Ops UI.** A web UI showing report status, the exceptions queue, reconstruction lookup, and the three-pillar dashboards (completeness / accuracy / timeliness); a non-engineer self-serve demo path. *Done when:* a non-engineer could check "did my trades report on time?" without touching a terminal.
- **Day 38 — SLOs + full CI matrix.** Numeric SLOs with error budgets for the three pillars; a holiday-calendar + T+1 deadline tracker with cutoff alerts; the full GitHub Actions matrix (unit / integration / contract / golden-file / regression / load / chaos); DB + middleware performance tuning.
- **Day 39 — Scale + capacity.** Push past 1M trades (load); the chaos matrix in CI; the capacity-planning write-up toward 600M/day (per-second rate, partition count,

consumer parallelism, daily Cassandra storage growth, where the design breaks first).

- **Day 40 — Portfolio polish.** README with CI/coverage/security badges; tidy the ADR log and RFC folder; runbooks an operator could follow at 3am; five blameless postmortems on file; record the **3-minute demo video** (ingest a trade → watch it report → reconstruct its life → trigger a break → watch recon catch it). ★ **Definition of done:** the repo is the proof — green CI across every test type, design-before-build evidence, live SLO dashboards, real postmortems, and a demo that closes the loop.
-

Compression notes (where the risk is)

These days are ambitious as single days; protect them or let them spill into the block's buffer:

- **Days 14-16** — XSD-valid transformation + the submission state machine + its proofs. The XSD work is fiddly; don't rush it.
- **Days 35-36** — the C++ rewrite + a *credible* benchmark. If time is tight, a smaller hot-path slice with an honest number beats a big rewrite with a hand-wavy one.
- **Day 39** — 1M+ load and the capacity write-up. The write-up (reasoning on paper) is the high-signal part and survives even if the live load run is partial.

If you need to shed scope rather than hours: drop Phase 9 (BTCA-lite) entirely, and treat the C++ rewrite (Phase 1') and the UI (Phase 13) as the first things to defer — the correctness story (Days 16-20) and reconstruction (Block 5) are worth far more in an interview than either.